

Name: A Raja

Reg No: 192365033

Course Code: CSA1718

PROGRAM – 7

Breadth First Search (BFS)

Aim

To traverse a graph using the **Breadth First Search (BFS)** algorithm.

Algorithm

1. Create a graph using an adjacency list.
2. Select the starting node.
3. Mark the starting node as visited and insert it into the queue.
4. Remove a node from the queue and visit it.
5. Add all unvisited adjacent nodes to the queue.
6. Repeat until the queue becomes empty.

Python Program

```
from collections import deque
```

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': ['F'],  
    'F': []  
}
```

```
visited = set()  
queue = deque()
```

```
start = 'A'
visited.add(start)
queue.append(start)

print("BFS Traversal:")

while queue:
    node = queue.popleft()
    print(node, end=" ")

    for neighbour in graph[node]:
        if neighbour not in visited:
            visited.add(neighbour)
            queue.append(neighbour)
```

Expected Output

BFS Traversal:
A B C D E F

PROGRAM – 8

Depth First Search (DFS)

Aim

To traverse a graph using the **Depth First Search (DFS)** algorithm.

Algorithm

1. Create a graph using an adjacency list.
2. Select the starting node.
3. Mark the current node as visited.
4. Visit the current node.
5. Recursively visit all unvisited adjacent nodes.
6. Continue until all nodes have been visited.

Python Program

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': ['F'],  
    'F': []  
}  
  
visited = set()  
  
def dfs(node):  
    if node not in visited:  
        print(node, end=" ")  
        visited.add(node)  
  
        for neighbour in graph[node]:  
            dfs(neighbour)  
print("DFS Traversal:")  
dfs('A')
```

Expected Output

DFS Traversal:

A B D E F C